

Санкт-Петербургский государственный университет

Морозко Иван Дмитриевич

Выпускная квалификационная работа

Реализация алгоритма поиска точных
запрещающих ограничений в платформе
Desbordante

Уровень образования: магистратура

Направление *09.04.04 «Программная инженерия»*

Основная образовательная программа *ВМ.5666.2024 «Программная инженерия»*

Научный руководитель:
к. ф.-м. н., старший научный сотрудник кафедры информационно-аналитических систем
Е. Г. Михайлова

Консультант:
к. ф.-м. н., ассистент кафедры информационно-аналитических систем Г. А. Чернышев

Рецензент:
ведущий эксперт ООО «СИГМА» Д. Ю. Гончаров

Санкт-Петербург
2026

Saint Petersburg State University

Ivan Morozko

Master's Thesis

Implementing exact denial constraint discovery algorithm in Desbordante platform

Education level: master

Speciality *09.04.04 "Software Engineering"*

Programme *BM.5666.2024 "Software Engineering"*

Scientific supervisor:
senior researcher, C.Sc. E. G. Mikhailova

Consultant:
assistant, C.Sc. G. A. Chernishev

Reviewer:
lead expert at LLC "SIGMA" D. Y. Hancharou

Saint Petersburg
2026

Оглавление

Введение	4
1. Постановка задачи	6
2. Обзор	7
2.1. Базовые определения	7
2.2. Обзор существующих алгоритмов	8
2.3. Алгоритм ECP/HEI-P	11
2.4. Платформа Desbordante	17
2.5. Алгоритм FastADC	17
3. Архитектура	19
4. Особенности реализации	24
4.1. Построение набора свидетельств	24
4.2. Перечисление покрытий набора свидетельств	25
4.3. Диаграмма классов	28
4.4. Интеграция в платформу Desbordante	29
5. Экспериментальное исследование	32
5.1. Методология	32
5.2. Результаты	34
5.3. Выводы	41
Заключение	43
Список литературы	44

Введение

К сожалению, данные, основанные на экспериментах из реального мира, зачастую оказываются неидеальными: в них могут содержаться ошибки, такие как пропущенные значения, неправильные значения и т. п. Недостоверная информация может привести к принятию неверного решения, поэтому очистке данных в современном мире уделяется особое значение [9].

В этом вопросе может помочь профилирование данных — процесс извлечения метаданных из набора данных [1]. В рамках профилирования исследуются различные виды формально задаваемых зависимостей и ограничений. Например, к числу наиболее известных относятся функциональные зависимости [14]. Благодаря профилированию данных можно узнать внутренние закономерности и, благодаря им, сделать выводы о возможных ошибках [8].

Одним из наиболее выразительных классов таких ограничений являются запрещающие ограничения (Denial Constraints). Каждое запрещающее ограничение (ЗО) представляет собой отношение между предикатами, указывающее, какие комбинации значений атрибутов являются несогласованными. За счёт этого ЗО обобщают ряд известных видов зависимостей и ограничений, включая функциональные зависимости, условные функциональные зависимости (Conditional Functional Dependencies, CFD), уникальные комбинации столбцов (Unique Column Combinations, UCC) и зависимости порядка (Order Dependencies, OD), а также позволяют выражать более сложные правила зависимостей данных [4, 5, 6].

Автоматический поиск ЗО является трудоёмкой задачей [2], однако высокая выразительность этого класса ограничений делает его полезным для задач очистки данных. Как следствие, в ряде современных систем ЗО используются в качестве языка для описания ограничений данных [7, 12]. Например, система HOLOCLEAN [15] принимает на вход наборы приближённых ЗО.

В рамках предыдущих работ [24, 25] мною был реализован и ин-

тегрирован в открытую платформу профилирования данных Desbor-dante [11] алгоритм поиска приближённых запрещающих ограничений FastADC [13], в том числе его многопоточная версия. Многопоточная реализация показала ускорение по времени в среднем в 2.64 раза и экономию по памяти в среднем в 13.05 раза по сравнению с оригинальной реализацией на Java [25].

Тем не менее, в ряде практических сценариев пользователям требуется поиск не приближённых, а точных запрещающих ограничений — то есть таких ЗО, которые не нарушаются ни на одной паре кортежей в наборе данных. Формально это соответствует установке порога ошибки равным нулю при использовании алгоритмов поиска приближённых ЗО. Однако алгоритм, специализированный на поиске точных ЗО, может быть более эффективен, чем алгоритм поиска приближённых ЗО с нулевым порогом ошибки, так как он может избегать вычислений, необходимых исключительно для поддержки приближённого режима [18].

В связи с этим было принято решение провести обзор существующих алгоритмов поиска точных ЗО, выбрать наиболее подходящий из них и реализовать соответствующее решение в платформе Desbor-dante. Это позволит пользователям получить более производительный инструмент.

1. Постановка задачи

Целью данной работы является разработка и реализация решения для поиска точных запрещающих ограничений в платформе Desbordante, дающего лучший результат по сравнению с алгоритмом FastADC при нулевом пороге ошибки. Для её достижения были сформулированы следующие задачи.

- Провести обзор существующих алгоритмов поиска точных запрещающих ограничений и выбрать наиболее подходящий. Детально изучить выбранный алгоритм.
- Разработать решение на основе выбранного алгоритма, адаптированное к платформе Desbordante.
- Реализовать разработанное решение в платформе Desbordante.
- Провести экспериментальное исследование: сравнить производительность реализованного решения с оригинальной реализацией выбранного алгоритма и с алгоритмом FastADC при нулевом пороге ошибки.

2. Обзор

2.1. Базовые определения

Опишем основные понятия, используемые для описания запрещающих ограничений и алгоритма их поиска.

Используются стандартные определения из реляционной модели данных: R — заголовок отношения; $r \in R$ — отношение¹; $s, t \in r$ — кортежи. Следующие определения предиката, точного, приближённого и минимального запрещающего ограничения, а также меры ошибки g_1 , приведены по работе [13].

Определение 1 (Предикат). Запрещающие ограничения определяются на основе предикатов. Каждый предикат p имеет форму $t.A_i \text{ op } s.A_j$, где $t, s \in r$, $t \neq s$ (различные кортежи), $A_i, A_j \in R$, и $\text{op} \in \{<, \leq, >, \geq, =, \neq\}$. Сравнение проводится по одному и тому же атрибуту, если $i = j$. В противном случае, оно проводится между двумя атрибутами.

Определение 2 (Запрещающие ограничения). ЗО φ на отношении r определяется следующим образом: $\forall t, s \in r, \neg(p_1 \wedge \dots \wedge p_m)$, где $p_1, \dots, p_m$ являются предикатами для кортежей t, s . Мы говорим, что φ удерживается на r , если для каждой пары кортежей $(t, s) \in r$ по крайней мере один из $p_1, \dots, p_m$ не удовлетворяется. Пара кортежей (t, s) нарушает φ , если пара удовлетворяет каждому $p_i : i \in [1, m]$.

Определение 3 (Точное запрещающее ограничение). Запрещающее ограничение является точным, если не существует ни одной пары кортежей $(t, s) \in r$, нарушающей φ .

Определение 4 (Мера ошибки). Значение меры ошибки g_1 для данного ЗО φ на r , обозначаемое как $g_1(\varphi, r)$, определяется как отношение числа пар кортежей, нарушающих φ , к общему числу пар кортежей в r . То есть, $g_1(\varphi, r) = \frac{|\{(t,s) | (t,s) \in r^2 \wedge (t,s) \text{ нарушает } \varphi\}|}{|r|^2 - |r|}$.

¹Алгоритм не требует отсутствия дубликатов, поэтому тут и далее можно рассматривать отношение как таблицу.

Определение 5 (Приближённое запрещающее ограничение). При заданном пороге ошибки ϵ , φ является приближённым ЗО на r , если и только если $g_1(\varphi, r) \leq \epsilon$.

Определение 6 (Минимальное запрещающее ограничение). ЗО φ является минимальным, если не существует отличного от φ ограничения φ' , такого что множество предикатов φ' является строгим подмножеством множества предикатов φ , и φ' удерживается на r .

Приведём несколько примеров ЗО, удерживающихся на таблице 1.

Таблица 1: Пример таблицы.

A	B	C
1	2	фрукт
2	1	овощ
2	4	овощ

(a) функциональная зависимость $A \rightarrow C$ может показать, что любые два кортежа, совпадающие на атрибуте A, совпадают на атрибуте C. (b) уникальная комбинация колонок (Unique Column Combination, UCC) может показать, что проекция по атрибутам A и B не имеет одинаковых кортежей. Оба этих ограничения можно выразить в терминах ЗО:

$$(a) \forall t, s \in r, \neg(t.A = s.A \wedge t.C \neq s.C)$$

$$(b) \forall t, s \in r, \neg(t.A = s.A \wedge t.B = s.B)$$

$$(c) \forall t, s \in r, \neg(t.A > s.B \wedge t.B \leq s.B)$$

Также ЗО (c) показывает, что возможность использования не только операторов равенства/неравенства, но и числового сравнения, позволяет выражать и более сложные зависимости между данными.

2.2. Обзор существующих алгоритмов

Разработка алгоритмов поиска точных и приближённых запрещающих ограничений ведётся с 2013 года с представления алгоритма Fast-DC [5], и постоянно открываются новые методы. Для оценки существующих алгоритмов поиска ЗО и выбора подходящего для реализации

алгоритма в платформе Desbordante, было выделено несколько критериев отбора.

Критерии отбора

- возможность поиска гетерогенных (cross-column) ЗО;

Возможность использовать предикаты между различными атрибутами позволит увеличить объём информации, которую мы способны получить из данных.

- нахождение минимальных ЗО;

Нахождение минимальных запрещающих ограничений предпочтительно, поскольку оно позволяет избежать избыточности. Алгоритмы, находящие все ЗО, во-первых, используют больше памяти, а во-вторых, затрудняют очистку данных. Если ЗО φ удерживается на r , и существует ЗО ξ , следующее из φ , то в проверке ξ при очистке данных нет смысла.

- наличие open-source реализации;

Наличие реализации с открытым кодом упрощает процесс реализации алгоритма на языке C++, и позволяет сравнить полученное решение с оригинальным.

- эффективность по сравнению с предыдущими.

Самый эффективный алгоритм является и самым предпочтительным для реализации. Зачастую каждое новое исследование в области алгоритмов поиска ЗО направлено на преодоление ограничений предшествующих подходов. Знак минуса в данном столбце таблицы означает что алгоритм был превзойдён по скорости более поздними разработками.

Оценка рассмотренных алгоритмов представлена на таблице 2. Алгоритмы представлены в порядке их выхода, от самого старого на первой строке, до state-of-the-art на последних. Алгоритм FastADC уже реализован в платформе Desbordante и отмечен жёлтым цветом.

Таблица 2: Оценка рассмотренных алгоритмов.

Алгоритм	Год	Гетерогенные ЗО	Минимальные ЗО	Открытый код	Эффективнее предыдущих
FastDC [5]	2013	+	+	+	—
Hydra [4]	2017	+	+	+	—
BFastDC [19]	2018	+	+	—	—
DCFinder [20]	2019	+	+	+	—
ADCMiner [3]	2020	+	+	+	—
ECP/HEI-P [18]	2022	+	+	+	+
ECP/HMMCS-P [18]	2022	+	+	+	+
FastADC [13]	2022	+	+	+	+

Таким образом, алгоритмами, удовлетворяющими всем выбранным критериям, стали ECP/HEI-P и ECP/HMMCS-P. Авторами этих алгоритмов была предоставлена открытая реализация для нахождения точных ЗО². Кроме того, в работе описано, каким образом предложенные подходы могут быть адаптированы для поиска приближённых запрещающих ограничений [18].

Несмотря на то, что оба алгоритма удовлетворяют сформулированным критериям, для реализации был выбран ECP/HEI-P. Данный выбор обусловлен результатами экспериментального исследования, приведённого авторами работы [18]. В работе отмечается, что ECP/HEI-P показывает лучшие результаты в случаях с большим числом предикатов и большим числом найденных ЗО. Такие случаи представляют особый интерес на реальных наборах данных, поскольку пространство поиска ЗО быстро растёт с увеличением числа атрибутов и порождённых ими предикатов.

Отметим, что разработка алгоритмов в данной области продолжается и после 2022 года. К примеру, в работе [21] предложен алгоритм Rapidash, ориентированный прежде всего на верификацию ЗО. Поиск ЗО в данном подходе осуществляется посредством последовательной верификации всевозможных кандидатов. А в работе [17] представлен подход к поиску ЗО с помощью аппаратного ускорения на ПЛИС.

²<https://github.com/eduardopena/fdcd>

2.3. Алгоритм ЕСР/НЕИ-Р

На текущий момент все алгоритмы поиска точных ЗО следуют схеме, состоящей из двух этапов [4, 5, 13, 18, 20]. На первом этапе по данному отношению r строится структура, называемая набором свидетельств (evidence set, НС), а на втором этапе происходит поиск покрытий НС.

Алгоритм ЕСР/НЕИ-Р тоже следует данной схеме. Первый этап называется конвейер контекстов свидетельств (Evidence Context Pipeline, ЕСР), а второй этап — гибридная инверсия свидетельств (Hybrid Evidence Inversion-Parallel, НЕИ-Р). Опишем их подробнее.

Конвейер контекстов свидетельств (ЕСР). В начале для построения набора свидетельств происходит построение так называемого пространства предикатов P — множества всех допустимых предикатов на r . Пространство предикатов строится, используя все операторы сравнения для числовых атрибутов, и операторы равенства/неравенства для категориальных. При этом используются только *сравнимые* атрибуты, что позволяет избегать неинтересных предикатов. Атрибут всегда считается сравнимым сам с собой. Для различных категориальных атрибутов сравнимость определяется по доле общих значений: пара атрибутов используется только в том случае, если эта доля не меньше заданного порога (30% по умолчанию). В свою очередь два числовых атрибута считаются сравнимыми, если их средние отличаются друг от друга не более чем на заданную долю. Например, при значении параметра по умолчанию — 0.1, меньшее среднее должно составлять не менее 90% от большего. Если это условие не выполняется, пара атрибутов не используется при построении пространства предикатов.

Пример пространства предикатов можно увидеть на таблице 3. Данное пространство предикатов построено для таблицы 1, обсуждённой в разделе 2.1.

Определение 7 (Свидетельство). Свидетельство $evi(t, s)$ для пары кортежей $(t, s) \in r^2$ представляет собой множество предикатов $p \subset P$, удовлетворяющих на паре кортежей (t, s) .

Таблица 3: Пространство предикатов для таблицы 1.

$p_1 : t.A = s.A$	$p_2 : t.A \neq s.A$
$p_3 : t.A > s.A$	$p_4 : t.A < s.A$
$p_5 : t.A \geq s.A$	$p_6 : t.A \leq s.A$
$p_7 : t.A = s.B$	$p_8 : t.A \neq s.B$
$p_9 : t.A > s.B$	$p_{10} : t.A < s.B$
$p_{11} : t.A \geq s.B$	$p_{12} : t.A \leq s.B$
$p_{13} : t.B = s.B$	$p_{14} : t.B \neq s.B$
$p_{15} : t.B > s.B$	$p_{16} : t.B < s.B$
$p_{17} : t.B \geq s.B$	$p_{18} : t.B \leq s.B$
$p_{19} : t.C = s.C$	$p_{20} : t.C \neq s.C$

Определение 8 (Набор свидетельств). НС определяется как множество свидетельств по всем парам кортежей: $\{evi(t, s) : (t, s) \in r^2\}$.

Авторы алгоритма отмечают что зачастую значительная часть пар кортежей в r порождает одно и то же свидетельство, и предлагают избегать поэлементного обхода всех $|r|^2$ пар, используя вместо этого промежуточную структуру, называемую контекстом свидетельства.

Определение 9 (Контекст свидетельства). Контекстом свидетельства называется тройка $ect = \langle t, tids, e \rangle$, где $t \in r$ — кортеж, $tids \subseteq r$ — множество кортежей, а $e \subseteq \text{НС}$ — свидетельство. Тройка ect интерпретируется как утверждение: “кортеж t в паре с любым кортежем $t' \in tids$ порождает свидетельство e ”.

Контексты свидетельств позволяют закодировать одной структурой сразу $|tids|$ пар кортежей, имеющих одинаковое свидетельство, что обеспечивает более компактное представление данных.

Построение набора свидетельств в ЕСП организовано следующим образом. Для каждого кортежа $t \in r$ запускается отдельный конвейер, состоящий из последовательности *стадий коррекции*. Изначально множество контекстов для t инициализируется одним контекстом $\langle t, r \setminus \{t\}, e_0 \rangle$, где e_0 — начальное свидетельство $\{p \in P \mid p.op \in \{\neq, >, \geq\}\}$.

Каждая стадия соответствует одной группе предикатов, то есть набору предикатов, заданных на одной и той же паре атрибутов. Стадия

коррекции получает на вход текущее множество контекстов и уточняет его относительно своей группы предикатов. Для каждого контекста $\langle t, tids, e \rangle$ множество $tids$ разбивается на части в зависимости от того, какой предикат из группы выполняется для t и $t' \in tids$. Для каждого такого подмножества формируется обновлённый контекст с соответствующим свидетельством. Таким образом, один контекст может быть разделён на несколько более точных контекстов, некоторые контексты могут исчезнуть, а уже корректные контексты могут остаться без изменений.

ЕСР выполняется параллельно, так как множества контекстов для коррекции для разных кортежей t независимы друг от друга. Коррекции, выполняемые в конвейере для кортежа t_i , не используют и не изменяют контексты, построенные для другого кортежа t_j . Поэтому обработку разных кортежей можно распределить между потоками: каждый поток запускает ЕСР для своего подмножества кортежей и строит локальные контексты свидетельств. Синхронизация требуется только на последнем шаге, когда полученные свидетельства собираются в общий набор свидетельств.

В рамках данной работы алгоритм ЕСР не реализован, так как построение набора свидетельств выполняется средствами ранее реализованного алгоритма FastADC. Обоснование такого решения приведено в разделе 3.

Гибридная инверсия свидетельств (HEI-P). Авторы статьи [5], положившей начало алгоритмам поиска запрещающих ограничений, ввели и доказали следующую теорему:

Определение 10 (Покрытие НС). Покрытие НС — это подмножество X множества P , такое что $\forall e \in \text{НС } X \cap e \neq \emptyset$.

Теорема 1 (Удержание точного ЗО). ЗО $\varphi = \neg(\overline{p_1} \wedge \dots \wedge \overline{p_m})$ удерживается на r , тогда и только тогда, когда множество $\{p_1, \dots, p_m\}$ является покрытием НС.

Авторы ЕСР/НЕI-Р используют эквивалентную формулировку этой задачи через понятие отрицательного покрытия набора свидетельств.

Определение 11 (Отрицательное покрытие НС). Множество предикатов $X \subseteq P$ называется *отрицательным покрытием* НС (Negative Cover), если не существует свидетельства, содержащего все предикаты из X : $\nexists e \in \text{НС} : X \subseteq e$.

Для множества предикатов $X = \{p_1, \dots, p_m\}$ обозначим через $\overline{X}$ множество инвертированных предикатов: $\overline{X} = \{\overline{p}_1, \dots, \overline{p}_m\}$.

Авторы работы [18] показывают что поиск точных запрещающих ограничений эквивалентен поиску отрицательных покрытий НС. Для этого достаточно доказать, что

$$\overline{X} \text{ покрытие НС} \iff X \text{ отрицательное покрытие НС.}$$

Действительно, рассмотрим произвольное свидетельство $e \in \text{НС}$. Так как свидетельство содержит все предикаты, истинные на некоторой паре кортежей, то для каждого предиката p из X в свидетельстве не могут одновременно содержаться p и $\overline{p}$. Более того, если $\overline{p} \notin e$, то на данной паре кортежей истинным является p , то есть $p \in e$. Следовательно,

$$\overline{X} \cap e = \emptyset \iff X \subseteq e.$$

Тогда

$$\begin{aligned} \overline{X} \text{ покрытие НС} &\iff \forall e \in \text{НС} : \overline{X} \cap e \neq \emptyset \\ &\iff \forall e \in \text{НС} : X \not\subseteq e \\ &\iff \nexists e \in \text{НС} : X \subseteq e \\ &\iff X \text{ отрицательное покрытие НС} \end{aligned}$$

Также авторами показано, что задача нахождения минимальных точных ЗО сводится к задаче перечисления минимальных отрицательных покрытий набора свидетельств [18]. Для этого алгоритм НЕI-Р использует две процедуры: индексированный поиск отрицательных покрытий (Indexed Negative Cover Search, INCS) и инверсию свидетельств

(Evidence Inversion, EI).

Индексированный поиск отрицательных покрытий. Это алгоритм поиска отрицательных покрытий, основанный на обходе дерева кандидатов в глубину. Вершиной дерева является тройка $(path, preds, E)$, где $path$ — множество предикатов для отрицательного покрытия, $preds$ — список предикатов, которые ещё могут быть добавлены к $path$, а E — множество свидетельств, всё ещё содержащих все предикаты из $path$. Иными словами, E содержит именно те свидетельства, которые пока не позволяют считать $path$ отрицательным покрытием.

Корень дерева соответствует начальному состоянию $(\emptyset, P, \text{НС})$, то есть пустому множеству выбранных предикатов, полному пространству предикатов P и исходному набору свидетельств НС.

На каждом шаге INCS выбирает очередной предикат $p \in preds$, добавляет его в $path' = path \cup \{p\}$ и строит новое множество свидетельств $E' = \{e \in E \mid path' \subseteq e\}$. Тем самым происходит переход от вершины $(path, preds, E)$ в вершину $(path', preds', E')$, где $preds'$ — обновлённый список предикатов. Например, если в $path$ был добавлен предикат $t.A > s.B$, то из $preds$ исключаются предикаты $t.A = s.B$, $t.A \neq s.B$, $t.A < s.B$, $t.A \leq s.B$ и $t.A \geq s.B$, поскольку такие предикаты приведут либо к не минимальным, либо к тривиальным ЗО. Если после добавления p множество E' становится пустым, то не существует свидетельства, содержащего все предикаты из $path \cup \{p\}$. Следовательно, найден кандидат для минимального отрицательного покрытия. Если же E' не пусто, поиск продолжается рекурсивно.

Минимальность проверяется после завершения обхода дерева. Это позволяет совершить обход дерева параллельно. Разные поддеревья могут обрабатываться в отдельных потоках, а найденные ими кандидаты объединяются в общее множество. После завершения параллельного поиска выполняется единая проверка минимальности.

Инверсия свидетельств. Это алгоритм перечисления отрицательных покрытий, изначально представленный в алгоритме Hydra [4].

В отличие от INCS, который строит кандидата путём рекурсивного добавления предикатов, EI обрабатывает свидетельства последовательно. Алгоритм поддерживает текущее множество кандидатных отрицательных покрытий X . При обработке очередного свидетельства e , проверяется, для каких кандидатов $x \in X$ выполняется $x \subseteq e$. Такой кандидат ещё не является отрицательным покрытием, поскольку найдено свидетельство, содержащее все его предикаты. Чтобы исправить такого кандидата, к x добавляется предикат p , не содержащийся в текущем свидетельстве: $x' = x \cup \{p\}$, $p \notin e$. Как и INCS, EI не добавляет предикаты из одной группы, чтобы избежать тривиальных ЗО. После обработки свидетельства e множество X заменяется полученными расширенными кандидатами, и алгоритм переходит к следующему свидетельству. Процесс повторяется до тех пор, пока не будут обработаны все свидетельства. Оставшиеся в X кандидаты являются отрицательными покрытиями.

Гибридная инверсия свидетельств. Наконец, HEI-P объединяет преимущества INCS и EI. Основная идея состоит в том, чтобы не запускать EI сразу на всём наборе свидетельств и всём пространстве предикатов, а сначала разбить исходную задачу на несколько меньших подзадач с помощью одного шага INCS.

На первом шаге INCS выбирается предикат $p \in P$. После этого формируется подзадача $(\{p\}, preds', \{e \in \text{HC} \mid p \in e\})$. После этого дальнейший поиск внутри полученной подзадачи выполняется алгоритмом EI. Если EI находит отрицательное покрытие X для уменьшенной подзадачи, то в исходной задаче ему соответствует кандидат $X \cup \{p\}$. Эти подзадачи независимы друг от друга, поэтому вызовы EI для разных p могут выполняться параллельно. Синхронизация требуется только при добавлении найденных кандидатов в общий результат.

После завершения всех параллельных вызовов EI выполняется единая проверка минимальности. Из полученного множества кандидатов удаляются те отрицательные покрытия, которые имеют собственное подмножество среди уже найденных покрытий. Оставшиеся множества

соответствуют минимальным отрицательным покрытиям, а значит, минимальным точным ЗО.

2.4. Платформа Desbordante

Desbordante — это высокопроизводительный инструмент для профилирования данных с открытым исходным кодом, разработанный на C++ [11]. За счет использования низкоуровневых оптимизаций, таких как векторизация вычислений и специализированные аллокатеры, Desbordante обладает высокой производительностью.

Этот инструмент обладает удобным пользовательским интерфейсом, включая Python-привязки и frontend-клиент.

В настоящее время Desbordante активно развивается, постоянно пополняясь реализациями новых алгоритмов.

2.5. Алгоритм FastADC

FastADC [13] это алгоритм поиска приближённых ЗО, реализованный в платформе Desbordante в рамках предыдущих работ [24, 25].

Для построения набора свидетельств FastADC использует промежуточную структуру — набор подсказок. Подсказка $clue(t, s)$ кодирует предикаты для кортежей t и s более компактно, чем свидетельство: одним битом для каждой категориальной пары атрибутов и двумя битами для каждой числовой пары. Каждой паре кортежей изначально присваивается начальная подсказка c_0 , которая затем корректируется для пар, чьи подсказки отличаются от c_0 ; такая коррекция эффективно реализуется с использованием индекса списка позиций (Position List Index, PLI).

В работе [25] был реализован параллельный алгоритм поиска подсказок. Таблица с данными разбивается на сегменты фиксированного размера, для каждой пары сегментов независимо строится частичный набор подсказок, после чего частичные результаты объединяются в глобальный.

Также в реализации FastADC в платформе Desbordante применяется оптимизация: выбор минимально достаточного типа данных для хранения подсказок в зависимости от размера пространства предикатов, что снижает затраты памяти и улучшает доступ к кешу процессора [25].

Для поиска приближённых ЗО FastADC использует алгоритм приближённой инверсии свидетельства (Approximate Evidence Inversion, AEI), представляющий собой расширение алгоритма EI, описанного в разделе 2.3. Поскольку текущая работа посвящена поиску точных ЗО, AEI не используется и в дальнейшем рассматриваться не будет.

Таким образом, оба рассмотренных алгоритма — ECP/HEI-P и FastADC — следуют одной и той же двухэтапной схеме: построение набора свидетельств и последующее перечисление покрытий. Такая структура открывает возможность комбинирования компонент из разных алгоритмов, что используется в предлагаемом решении, архитектура которого представлена далее.

3. Архитектура

Как было показано в разделе 2.3, алгоритмы поиска точных 3О естественно разделяются на два этапа: построение набора свидетельств и последующее перечисление покрытий этого набора. Между этими этапами существует явно заданный интерфейс, не зависящий от внутренних деталей того, каким именно способом был построен набор свидетельств.

В начале строится пространство предикатов, которое определяется исключительно таблицей с данными, опциональным параметром, разрешающим использование гетерогенных предикатов и порогом сравнимости атрибутов (раздел 2.3). Затем каждая пара кортежей преобразуется в свидетельство — множество предикатов из P , представляемое битовой шкалой длины $|P|$. Одинаковые свидетельства агрегируются, поэтому результатом этапа построения свидетельств является ассоциативный массив $evi \rightarrow cnt$, где cnt — число пар кортежей, породивших данное свидетельство. Этап перечисления покрытий получает на вход этот набор свидетельств и пространство предикатов. Благодаря этому ЕСР и FastADC могут использоваться как взаимозаменяемые способы построения набора свидетельств, а HEI-P и AEI — как взаимозаменяемые способы перечисления покрытий. В данной работе это свойство используется для построения комбинированного решения: построение набора свидетельств переиспользуется из FastADC, нахождение точных 3О реализуется через HEI-P.

Более подробно компоненты решения можно рассмотреть на UML-диаграмме на Рис. 1.

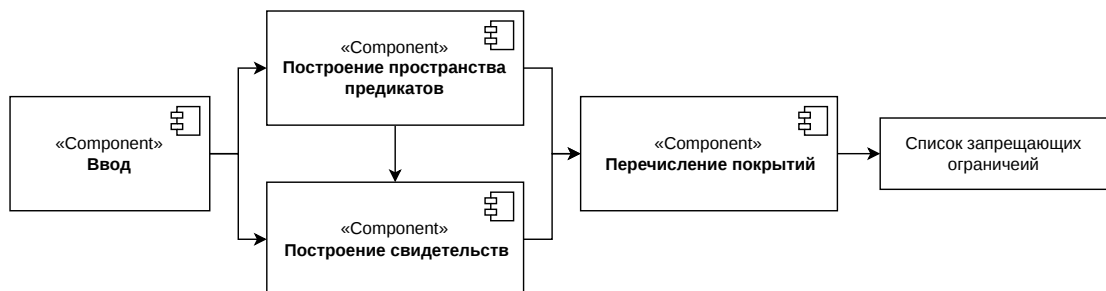


Рис. 1: UML-диаграмма компонент решения.

Ввод. Компонента приёма входных данных и параметров от пользователя. Реализована авторами Desbordante и имеет гибкую настройку, позволяющую определять тип и число входных данных. Для поиска ЗО данная компонента принимает от пользователя саму таблицу данных и набор конфигурационных параметров, таких как порог ошибки ϵ (по умолчанию равный нулю), порог сравнимости атрибутов, флаг разрешения гетерогенных предикатов и число потоков.

Построение пространства предикатов. Использует таблицу с данными, анализирует сравнимость атрибутов для определения допустимых предикатов. Формирует множество предикатов P , их числовые идентификаторы, и информацию об их группах: для каждого предиката — битовая шкала всех предикатов из той же группы.

Построение свидетельств. Использует предикаты и таблицу данных. Создаёт ассоциативный массив $evi \rightarrow cnt$, где свидетельство представлено битовой шкалой длины $|P|$, а cnt — число пар кортежей, породивших данное свидетельство.

Перечисление покрытий. Принимает набор свидетельств и пространство предикатов, возвращает список минимальных отрицательных покрытий, конвертируемых в объекты `DenialConstraint`.

При выборе способа построения набора свидетельств рассматривались два варианта: реализовать ЕСР по описанию из [18] или переиспользовать уже существующую реализацию построения набора свидетельств из FastADC. Для обоснования выбора был проведён небольшой эксперимент на тестовом стенде, описанном в таблице 5, с использованием наборов данных из таблицы 4. Для каждого набора данных выполнялось 10 запусков, после чего вычислялось среднее время работы с доверительным интервалом 95%. Подробнее с методологией тестирования можно ознакомиться в разделе 5.1.

В эксперименте сравнивалось время построения набора свидетельств в оригинальной Java-реализации ЕСР/HEI-P и в оригинальной Java-

реализации FastADC. Результаты представлены на рис. 2.

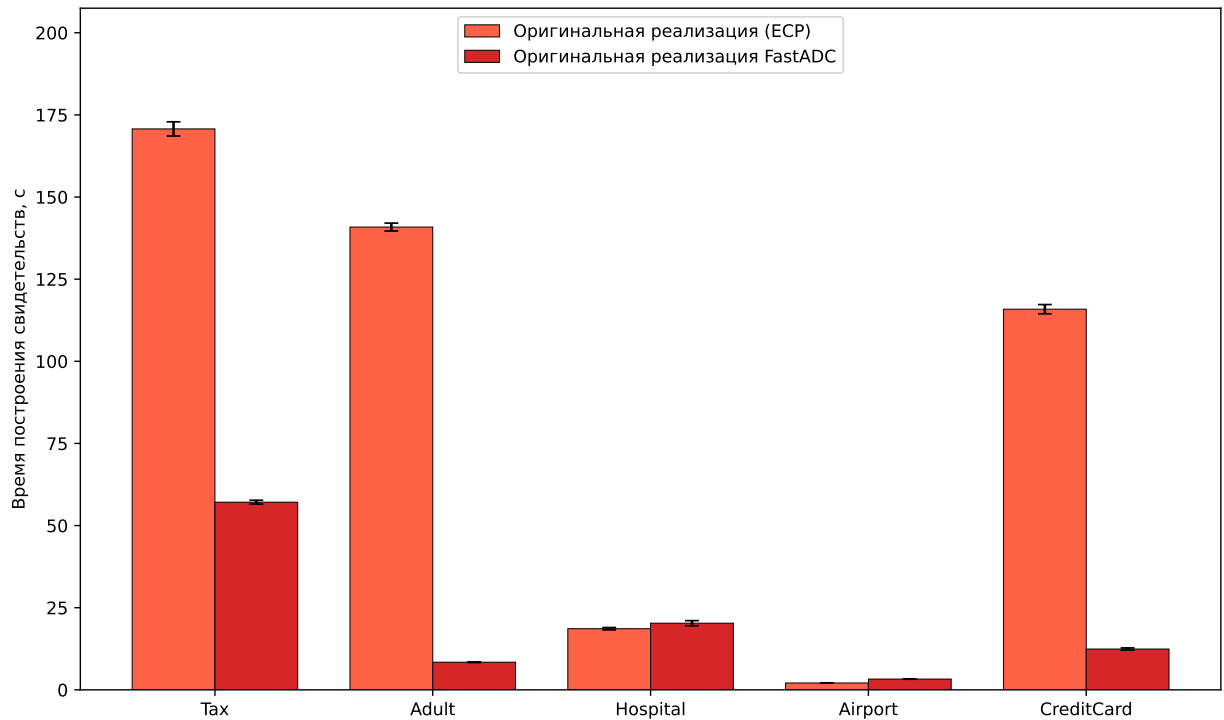


Рис. 2: Сравнение времени построения набора свидетельств: оригинальная Java-реализация ECP и оригинальная Java-реализация FastADC.

Полученные результаты показывают, что на небольших наборах данных преимущество FastADC выражено не всегда. К примеру, на наборе данных Airport FastADC оказался медленнее оригинального ECP в 1.56 раза. Однако на более сложных наборах данных преимущество реализации FastADC становится существенным: максимальное ускорение составило 16,74 раза, а среднее ускорение по всем рассмотренным наборам данных — 6,12 раза.

На основании этих результатов было принято решение использовать уже существующий механизм построения набора свидетельств из FastADC. Переиспользование реализации позволит не только избежать повторной реализации этапа построения НС, но и получить выигрыш по времени на больших наборах данных.

Главный сценарий работы. Поскольку поиск приближённых ЗО при пороге ошибки $\epsilon = 0$ совпадает с поиском точных ЗО, решение для поиска ЗО в Desbordante представлено единым пользовательским ин-

терфейсом, а выбор конкретного метода выполняется автоматически в зависимости от значения порога: при $\epsilon = 0$ запускается реализованный в данной работе HEI-P, при $\epsilon > 0$ — ранее реализованный AEI. Полученное решение было названо HybridDC. Решение интегрировано в платформу через стандартный интерфейс **Algorithm** и доступно из Python-привязок. UML-диаграмма активностей, демонстрирующая сценарий работы решения, приведена на Рис. 3.

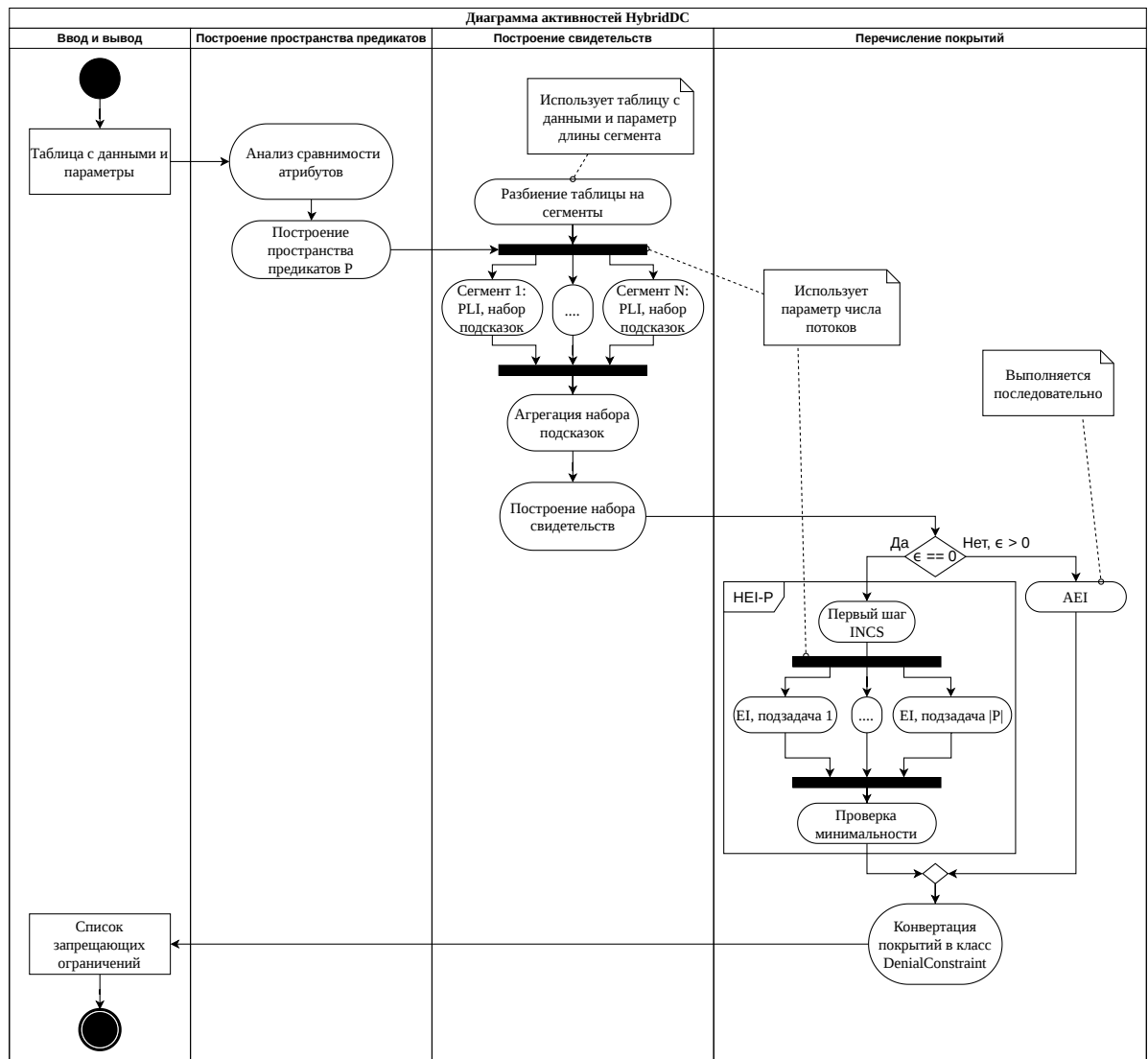


Рис. 3: UML-диаграмма активностей, описывающая работу решения.

Основные шаги:

1. Пользователь подаёт на вход таблицу с данными и параметры,

такие как порог сравнимости атрибутов, число потоков и т.п. Подробнее описание параметров представлено в разделе 4.4.

2. По таблице данных и части параметров строится пространство предикатов P с информацией об их группах.
3. Таблица разбивается на сегменты фиксированной длины. Для каждого сегмента параллельно строится индекс списка позиций (PLI) и частичный набор подсказок. Частичные результаты агрегируются в глобальный набор подсказок, который одним проходом преобразуется в набор свидетельств.
4. По значению порога ошибки выбирается алгоритм перечисления покрытий: HEI-P при $\epsilon = 0$, AEI при $\epsilon > 0$. Алгоритм получает на вход набор свидетельств и пространство предикатов.
5. Выбранный алгоритм возвращает список минимальных покрытий, которые конвертируются в объекты `DenialConstraint` и возвращаются пользователю.

Таким образом, в данном разделе была определена архитектура HybridDC: состав основных компонентов, порядок их вызова и интерфейс между построением набора свидетельств и перечислением покрытий. В следующем разделе будет описана интеграция в платформу Desbor-dante и описание какими конкретно классами и структурами данных реализуются эти компоненты.

4. Особенности реализации

Решение оформлено в виде нового алгоритма HybridDC, использующего существующие классы FastADC для построения набора свидетельств и новый класс *HEIInverter* для перечисления точных ЗО. При нулевом пороге ошибки HybridDC выполняет инверсию через *HEIInverter*; при положительном пороге — через ранее реализованный класс *ApproxEvidenceInverter* из FastADC.

4.1. Построение набора свидетельств

Этап построения набора свидетельств переиспользует классы, реализованные в рамках предыдущих работ [24, 25]. Изначально они не зависели от конкретного способа дальнейшего использования набора свидетельств, что позволило интегрировать их к новому алгоритму без изменений.

Используются следующие классы:

- *PredicateBuilder* — строит пространство предикатов. На выходе предоставляет список объектов *Predicate*;
- *PliShardBuilder* — разбивает таблицу на сегменты и строит PLI для каждого сегмента;
- *EvidenceSetBuilder* — параллельно строит набор подсказок по сегментам PLI, после чего конвертирует его в набор свидетельств.

В результате получается набор свидетельств в виде ассоциативного массива свидетельство $\rightarrow cnt$, а также пространство предикатов с информацией о группах. Эти структуры передаются на следующий этап.

Поскольку реализация и оптимизация этих классов подробно описаны в предыдущей работе, здесь они не рассматриваются повторно [25].

4.2. Перечисление покрытий набора свидетельств

Этот этап реализован в виде класса *HEIInverter*. Его основной метод *Run* принимает на вход набор свидетельств и пространство предикатов с информацией о группах, реализует алгоритм HEI-P (раздел 2.3) и возвращает список точных ЗО.

Свидетельство представляется фиксированной битовой шкалой длины 128. Тем самым на пространство предикатов накладывается соответствующее ограничение. В случае его превышения алгоритм завершается с сообщением о том, что пространство предикатов слишком велико для текущей реализации.

Работа метода *Run* состоит из нескольких последовательных стадий. Рассмотрим их подробнее.

Минимизация набора свидетельств. Если свидетельство e_1 является подмножеством свидетельства e_2 , то e_1 можно исключить из набора: любое отрицательное покрытие, нарушающее e_1 , нарушает и e_2 , и наоборот. Минимизация выполняется с помощью префиксного дерева *HEIBitSetTrie*.

Класс *HEIBitSetTrie* хранит множество битовых шкал и поддерживает операции вставки, удаления и проверки, есть ли в дереве шкала, являющаяся подмножеством или надмножеством данной битовой шкалы. Дерево устроено как префиксное представление множества установленных битов. Каждая битовая шкала записывается в дерево как возрастающая последовательность номеров её единичных битов. Например, шкала с установленными битами $\{3, 7, 15\}$ соответствует пути $3 \rightarrow 7 \rightarrow 15$ от корня. Благодаря этому при поиске подмножества для запроса достаточно рассматривать только ветви, номера которых присутствуют в запросе.

Сортировка предикатов. Для каждого предиката строится список идентификаторов свидетельств, в которые он входит, после чего предикаты сортируются по возрастанию по длине этого списка. При равен-

стве числа свидетельств приоритет получают предикаты между одинаковыми атрибутами. Такой порядок делает обход дерева кандидатов более удачным, поскольку при выполнении INCS множество оставшихся свидетельств чаще становится пустым на ранних шагах.

Параллельный цикл. По отсортированному списку предикатов запускается параллельный цикл. Для каждого предиката p выполняется один шаг алгоритма INCS, после чего для полученной подзадачи запускается внутренняя инверсия свидетельств алгоритмом EI.

Работа между потоками распределяется динамически. Вместо статического разбиения списка предикатов на равные части используется общий атомарный счётчик: каждый поток получает из него номер следующего ещё не обработанного предиката. Такой подход важен, поскольку время обработки разных предикатов может существенно отличаться. Для одних предикатов работа завершается почти сразу, например если предикат не входит ни в одно свидетельство. Для других требуется полноценный запуск внутреннего алгоритма EI. Поэтому при статическом разбиении часть потоков могла бы завершить свою долю работы раньше и простаивать, пока другие продолжают обрабатывать более трудные предикаты.

Синхронизация через мьютекс используется только при добавлении найденных кандидатов в общий список отрицательных покрытий. Эта критическая секция мала по сравнению с временем обработки одного предиката.

Финальная проверка минимальности. Все собранные кандидаты сначала сортируются по возрастанию размера покрытия, то есть по числу предикатов в нём.

Для каждого кандидата проверяется, существует ли среди уже сохранённых покрытий его подмножество. Если такое подмножество найдено, кандидат не является минимальным и отбрасывается. Иначе он добавляется в префиксное дерево *HEIBitSetTrie*, в котором хранятся уже найденные минимальные покрытия меньшего размера. Такой по-

рядок обработки позволяет выполнять проверки внутри одной группы параллельно. Действительно, если одно покрытие строго содержится в другом, то оно обязательно имеет меньший размер. Следовательно, покрытие того же размера не может быть собственным подмножеством текущего кандидата. Поэтому кандидаты одинакового размера можно проверять независимо друг от друга на неизменной структуре дерева. После завершения параллельных проверок оставшиеся кандидаты последовательно добавляются в дерево, и обработка переходит к следующей группе.

По завершении этой стадии оставшиеся минимальные покрытия конвертируются в объекты *DenialConstraint*.

Реализация HEI-P была оптимизирована с использованием профилирования. Применены следующие техники:

- узлы дерева размещаются в одном непрерывном массиве, а потомки задаются индексами этого массива. Такая организация уменьшает накладные расходы при создании узлов и улучшает локальность доступа к памяти: последовательно созданные узлы оказываются в близких участках памяти, поэтому при обходе дерева процессор чаще использует уже загруженные данные;
- каждый узел хранит первые четыре ссылки на потомков непосредственно внутри себя, а если потомков больше, для них используется хранилище в динамической памяти. Данная схема реализована через структуру данных `boost::container::small_vector`. На практике у большинства узлов число потомков невелико, поэтому обычно информация о потомках находится в той же области памяти, что и сам узел. Это уменьшает число дополнительных обращений к памяти при переходе от родителя к потомку;
- битовые шкалы, хранящиеся в дереве, вынесены в отдельный массив, а в самом узле остаётся только флаг наличия сохранённой шкалы и информация о потомках. Благодаря этому размер узла уменьшается, а данные, необходимые для основных операций об-

хода дерева, располагаются компактнее. Полная битовая шкала читается только в тех случаях, когда она действительно требуется, например при вставке или финальном обходе дерева;

- финальная проверка минимальности выполняется параллельно группами по размеру покрытия, то есть по числу предикатов в нём. В оригинальной реализации проверка минимальности была выполнена последовательным обходом всех найденных кандидатов: каждый кандидат временно удалялся из дерева *HEIBitSetTrie*, после чего проверялось, содержит ли дерево его подмножество. Если такое подмножество существовало, кандидат отбрасывался как неминимальный, затем кандидат снова добавлялся в дерево.

4.3. Диаграмма классов

На Рис. 4 представлена UML-диаграмма классов реализованного решения.

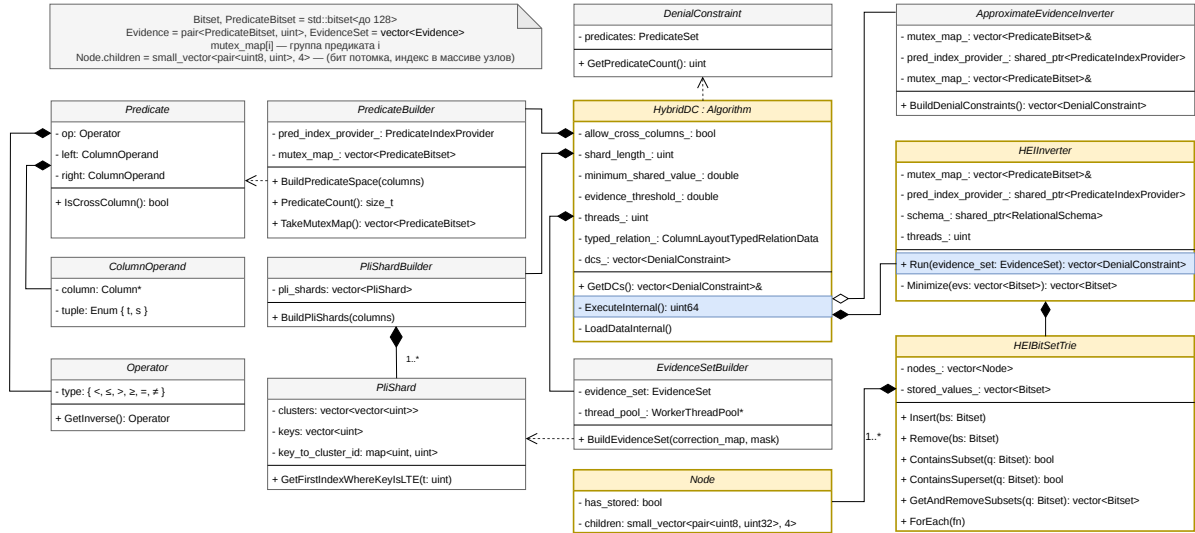


Рис. 4: UML-диаграмма классов алгоритма HybridDC.

Серым цветом выделены классы, переиспользованные из FastADC без изменений. Они отвечают за построение пространства предикатов, индексов списка позиций, набора свидетельств и поиск приближённых

Жёлтым цветом выделены классы, реализованные в рамках данной работы: *HybridDC* — основной класс алгоритма, координирующий выполнение всех этапов. *HEIInverter* — класс, реализующий HEI-P-инверсию набора свидетельств. *HEIBitSetTrie* — внутренняя структура данных *HEIInverter*, поддерживающая необходимые операции над множеством битовых шкал.

Синим цветом выделена основная функция *Run*, реализующая алгоритм HEI-P.

4.4. Интеграция в платформу Desbordante

Решение интегрировано в платформу Desbordante через систему привязок Python, реализованную с помощью библиотеки `pybind11` [16]. Алгоритм HybridDC доступен из Python через модуль `dc.algorithms` пакета `desbordante`. Основной интерфейс привязок включает:

- `execute` — запуск алгоритма с заданными параметрами;
- `get_dcs` — получение списка найденных запрещающих ограничений.

Параметры, задаваемые через систему опций Desbordante:

- `table` — входной файл с данными;
- `threads` — число потоков для параллельного выполнения;
- `shard_length` — длина сегмента для разбиения данных;
- `comparable_threshold` — порог сравнимости числовых атрибутов (Раздел 2.3);
- `minimum_shared_value` — порог сравнимости категориальных атрибутов (Раздел 2.3);
- `evidence_threshold` — порог ошибки: при нуле используются точные ЗО, при положительном значении — приближённые ЗО.

В Листинге 1 приведён пример использования алгоритма HybridDC. В данном примере в качестве входных данных используется таблица 1, сохранённая в файле `data.csv`; затем запускается поиск точных ЗО и выводятся найденные ограничения. Пример вывода программы приведён в Листинге 2.

```
1 import desbordante
2
3 algo = desbordante.dc.algorithms.HybridDC()
4 algo.load_data(table=("data.csv", ",", True))
5 algo.execute(
6     threads=8,
7     shard_length=0,
8     comparable_threshold=0.1,
9     minimum_shared_value=0.3,
10    evidence_threshold=0.0
11 )
12
13 dcs = algo.get_dcs()
14 print(f"\nFound {len(dcs)} denial constraints:\n")
15 for i, dc in enumerate(dcs, 1):
16     print(f"{i}. {dc}")
```

Листинг 1: Пример использования алгоритма HybridDC из Python.

Found 17 denial constraints:

1. $\neg\{ t.A \neq s.B \wedge t.B \geq s.B \wedge t.C \neq s.C \}$
2. $\neg\{ t.A \leq s.A \wedge t.A \geq s.B \wedge t.B \leq s.B \}$
3. $\neg\{ t.A \geq s.A \wedge t.A \neq s.B \wedge t.C \neq s.C \}$
4. $\neg\{ t.A \neq s.A \wedge t.A \neq s.B \wedge t.B \geq s.B \}$
5. $\neg\{ t.A > s.B \wedge t.C \neq s.C \}$
6. $\neg\{ t.A > s.A \wedge t.A \neq s.B \}$
7. $\neg\{ t.A \neq s.A \wedge t.A > s.B \}$
8. $\neg\{ t.A < s.B \wedge t.B \geq s.B \}$
9. $\neg\{ t.A > s.B \wedge t.B \leq s.B \}$
10. $\neg\{ t.B == s.B \}$
11. $\neg\{ t.A \neq s.A \wedge t.C == s.C \}$
12. $\neg\{ t.A == s.A \wedge t.C \neq s.C \}$
13. $\neg\{ t.A \leq s.B \wedge t.B \geq s.B \wedge t.C == s.C \}$
14. $\neg\{ t.A \geq s.B \wedge t.B \leq s.B \wedge t.C == s.C \}$
15. $\neg\{ t.A == s.A \wedge t.A \leq s.B \wedge t.B \geq s.B \}$
16. $\neg\{ t.A == s.B \wedge t.C == s.C \}$
17. $\neg\{ t.A == s.A \wedge t.A == s.B \}$

Листинг 2: Пример вывода алгоритма HybridDC для таблицы 1.

В полученном выводе присутствуют ограничения, соответствующие примерам из раздела 2.1. Например, ограничение 12 имеет вид $\neg(t.A = s.A \wedge t.C \neq s.C)$, то есть выражает функциональную зависимость $A \rightarrow C$, приведённую в примере (а). Ограничение 9 совпадает с примером (с): $\neg(t.A > s.B \wedge t.B \leq s.B)$.

Отдельно стоит отметить пример (b), где уникальная комбинация столбцов (A, B) была записана как $\neg(t.A = s.A \wedge t.B = s.B)$. В выводе HybridDC вместо неё присутствует более сильное минимальное ограничение 10: $\neg(t.B = s.B)$. Это означает, что в данной таблице уже один атрибут B является уникальным. Поэтому УСС на паре атрибутов (A, B) не является минимальным и не выводится алгоритмом.

5. Экспериментальное исследование

Для оценки производительности полученной реализации было проведено экспериментальное исследование. Его целью было ответить на следующие исследовательские вопросы:

- **RQ1:** *Насколько быстрее предложенное решение по сравнению с оригинальной реализацией ECP/HEI-P?*
- **RQ2:** *Позволяет ли предложенное решение сократить потребление памяти по сравнению с оригинальной реализацией ECP/HEI-P?*
- **RQ3:** *Насколько быстрее предложенное решение по сравнению с реализацией FastADC при нулевом пороге ошибки?*
- **RQ4:** *Оправдан ли предложенный подход по сравнению с существующим решением через использование FastADC при нулевом пороге ошибки?*

5.1. Методология

В экспериментах измерялись время построения набора свидетельств, время перечисления покрытий набора свидетельств, общее время выполнения и затраты памяти на наборах данных, представленных в таблице 4.

Таблица 4: Характеристики наборов данных, использовавшихся для проведения экспериментов.

Название	Атрибуты	Строки	Размер	# ЗО
Tax	15	99 904	7,0 Мб	185 561
Adult	15	32 561	3,7 Мб	1 450 420
Hospital	15	100 147	26 Мб	565
Airport	12	24 580	2,3 Мб	2 484
CreditCard	10	30 000	924 Кб	147 773

Обе реализации выводят время работы соответствующих этапов в стандартный поток вывода без учёта времени обработки входного фай-

ла. Эти значения использовались в качестве показателя производительности по времени. Память измерялась на отдельных запусках, где каждые 100 миллисекунд собиралась информация о затратах памяти командой `ps -o rss= -p $PID`. Для каждого набора данных проводилось десять запусков, из полученных данных вычислялось среднее значение с доверительным интервалом 95%.

Реализация в Desbordante компилировалась с флагом оптимизации `-O3`. Авторами оригинальной реализации не была предоставлена дополнительная информация о конфигурации виртуальной машины Java, поэтому была использована LTS-версия JDK с настройками виртуальной машины по умолчанию. Такой выбор согласуется с результатами работы [10]: авторы Desbordante исследовали влияние способов настройки Java-реализации, включая изменение параметров JVM и смену JVM, и пришли к выводу, что такие изменения не позволяют устойчиво улучшить производительность по сравнению с настройками по умолчанию.

Эксперименты проводились на тестовом стенде, представленном в таблице 5, без других ресурсоёмких процессов. Тестовый стенд представляет собой виртуальную машину KVM, размещённую на ненагруженном выделенном сервере. Хост-машина настроена администратором на режим максимальной производительности. Между запусками очищались кеши путём записи 3 в файл `/proc/sys/vm/drop_caches`. Файл подкачки на стенде отсутствует и не требовал отключения. Также на тестовом стенде была отключена поддержка одновременной многопоточности (Simultaneous Multithreading [22, 23]), поэтому каждое логическое ядро гостевой системы соответствовало ровно одному физическому ядру. Управление частотой процессора было недоступно из гостевой ОС.

Для каждой из реализаций использовалось 8 потоков, привязанных через `taskset` к ядрам с номерами от 0 до 7. Для Java-реализации, использующей `ForkJoinPool` для параллельного построения набора подсказок, явно задавался уровень параллелизма через параметры виртуальной машины:

- `-XX:ActiveProcessorCount=8` для указания числа доступных про-

цессоров;

- `-Djava.util.concurrent.ForkJoinPool.common.parallelism=8`
для задания размера общего набора исполнителей.

Таблица 5: Характеристики тестового стенда.

Компонент	Спецификация
Процессор	AMD EPYC 7713 64-Core Processor
Оперативная память	58 GiB (параметры подсистемы памяти на гостевом уровне недоступны)
Запоминающее устройство	499 GB virtio-блочное устройство (модель физического накопителя на гостевом уровне недоступна)
Гипервизор	KVM
Операционная система	Ubuntu 22.04.5 LTS
Ядро Linux	5.4.210-39.1
Версия OpenJDK	21.0.10 2026-01-20 LTS
Версия GCC	11.4.0

Чтобы избежать многочасовых ожиданий, для запуска FastADC устанавливалось ограничение времени в 60 минут, поскольку этот алгоритм спроектирован для поиска приближённых ЗО и при $\epsilon = 0$ выполняет ряд дополнительных операций, не требующихся для точного случая. Если первый запуск на наборе данных превышал ограничение времени, все десять запусков для этого набора пропускались целиком — такая ситуация отмечена на графиках красной полосой с пометкой TIMEOUT.

5.2. Результаты

Сравнение с оригинальной реализацией ECP/HEI-P. Результаты сравнения с оригинальной реализацией на языке Java представлены на Рис. 5–8.

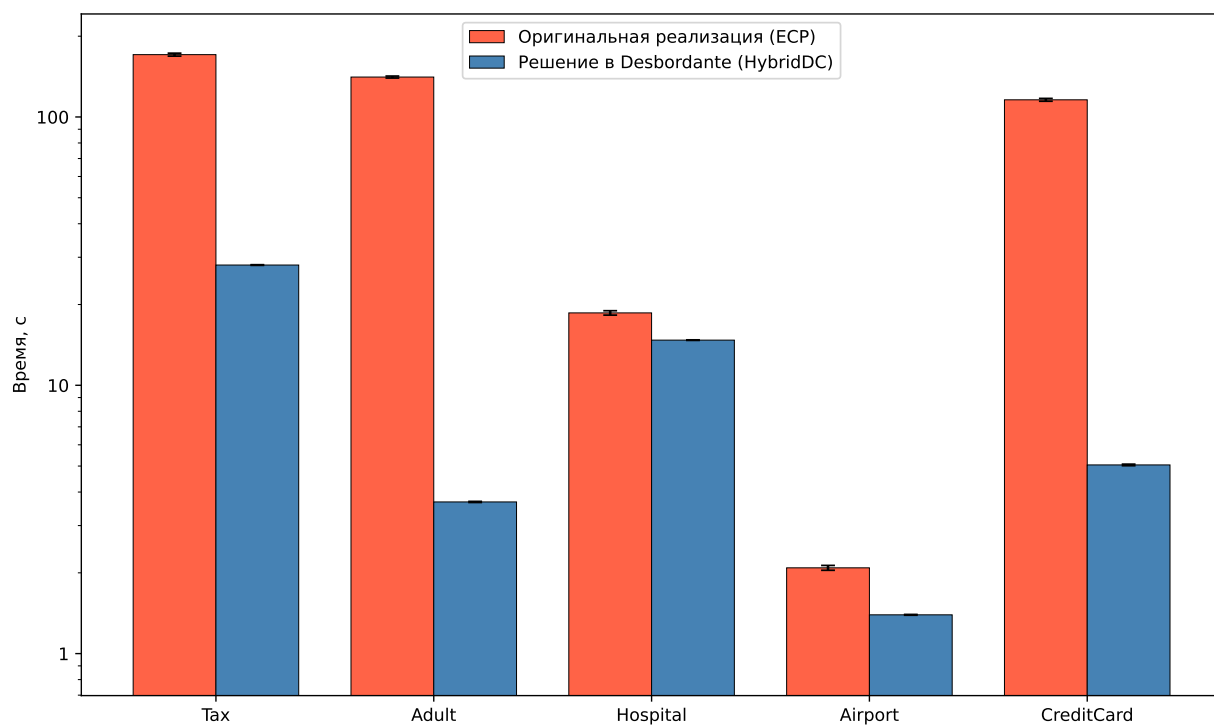


Рис. 5: Время построения набора свидетельств: сравнение с оригинальной реализацией.

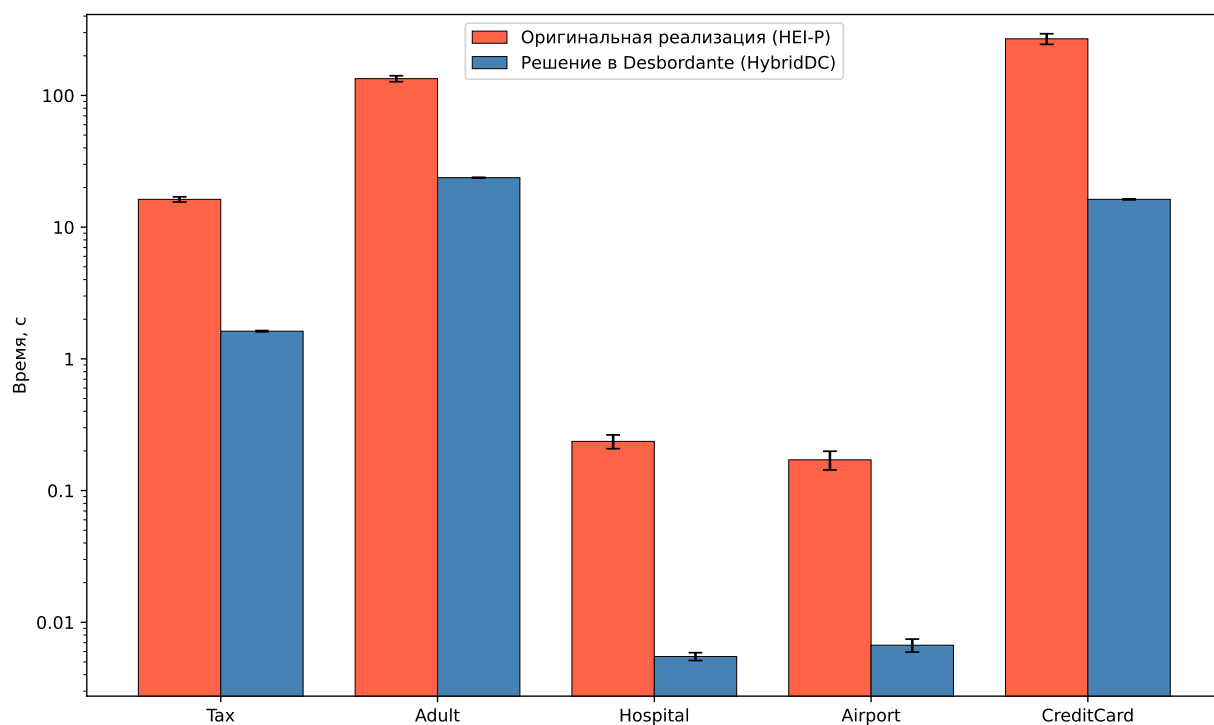


Рис. 6: Время перечисления покрытий: сравнение с оригинальной реализацией.

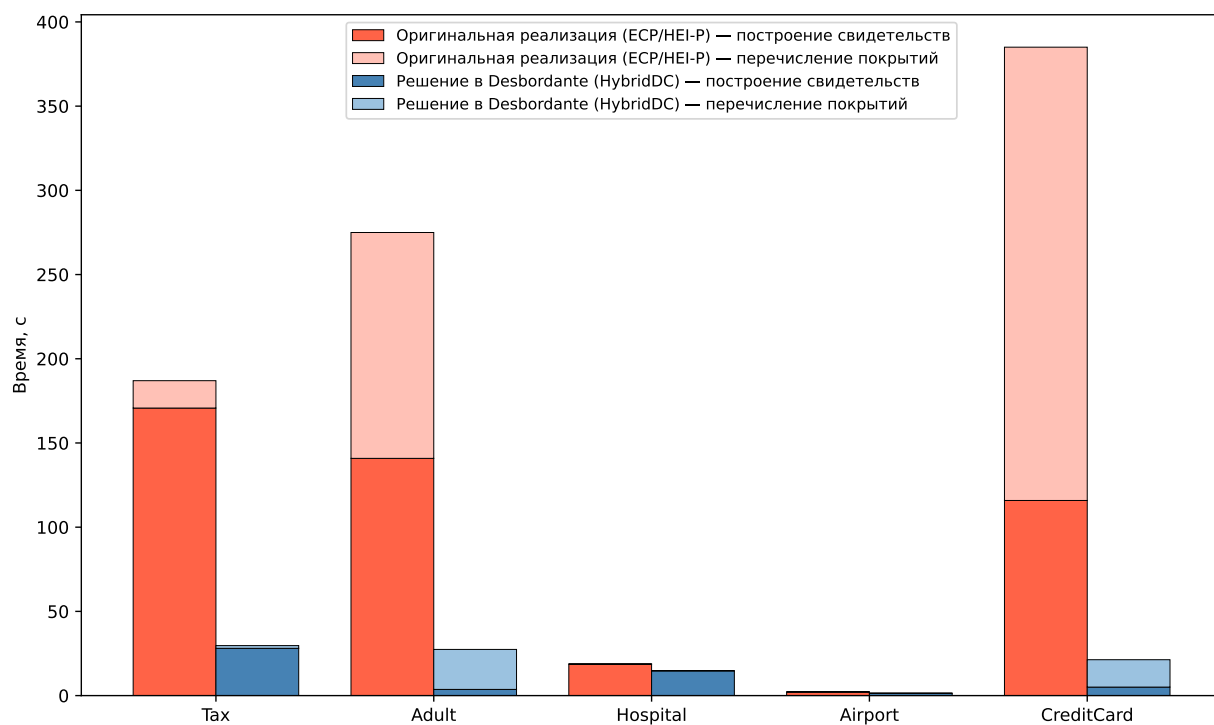


Рис. 7: Суммарное время выполнения: сравнение с оригинальной реализацией. Время представлено линейной шкалой. Каждая колонка разбита на два участка: нижний — время построения набора свидетельств, верхний — время перечисления покрытий.

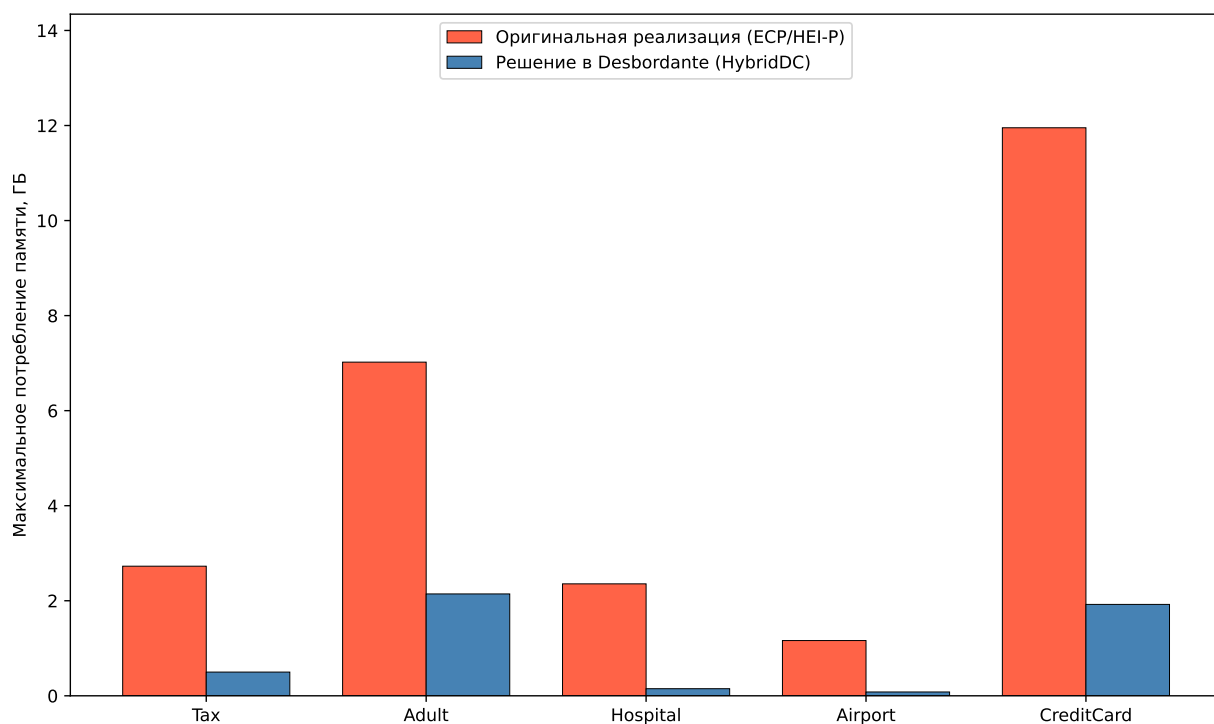


Рис. 8: Максимальное потребление памяти: сравнение с оригинальной реализацией.

Предложенная реализация HybridDC показала ускорение на всех пя-

ти наборах данных. По суммарному времени выполнения (Рис. 7) ускорение составило от 1,28 раза на наборе данных Hospital до 18,07 раза на наборе данных CreditCard. Среднее ускорение по суммарному времени составило 7,46 раза.

Отдельно стоит рассмотреть два этапа работы алгоритма. На этапе построения набора свидетельств (Рис. 5) ускорение составило от 1,26 раза на наборе Hospital до 38,33 раза на наборе Adult, в среднем — 14,02 раза.

На этапе перечисления покрытий набора свидетельств (Рис. 6) ускорение оказалось ещё более выраженным: от 5,65 раза на наборе Adult до 42,96 раза на наборе Hospital. В среднем ускорение этого этапа составило 20,15 раза.

Экономия по максимальному потреблению памяти (Рис. 8) составила от 3,28 раза на наборе Adult до 15,76 раза на наборе Hospital, в среднем — 9,03 раза.

Более подробно с численными результатами можно ознакомиться в таблице 6. В таблице приведены отношения средних значений по десяти запускам: ускорение построения набора свидетельств, ускорение перечисления покрытий, ускорение суммарного времени выполнения и экономия максимального потребления памяти. Минимальные значения ускорения и экономии памяти выделены светло-красным цветом, максимальные значения — светло-зелёным.

Таблица 6: Ускорение HybridDC относительно оригинальной реализации ECP/HEI-P.

Набор	# ЗО	Построение НС	Поиск покрытий	Общее время	Память
Tax	185 561	×6,08	×10,02	×6,30	×5,46
Adult	1 450 420	×38,33	×5,65	×10,03	×3,28
Hospital	565	×1,26	×42,96	×1,28	×15,76
Airport	2 484	×1,50	×25,55	×1,61	×14,44
CreditCard	147 773	×22,95	×16,55	×18,07	×6,22
Среднее	—	×14,02	×20,15	×7,46	×9,03

Сравнение с FastADC при нулевом пороге ошибки. Результаты сравнения с FastADC при нулевом пороге ошибки представлены на Рис. 9 и Рис. 10. Для этого сравнения не приводятся графики времени построения набора свидетельств и максимального потребления памяти, поскольку оба показателя в основном определяются способом построения набора свидетельств, который совпадает у обоих алгоритмов.

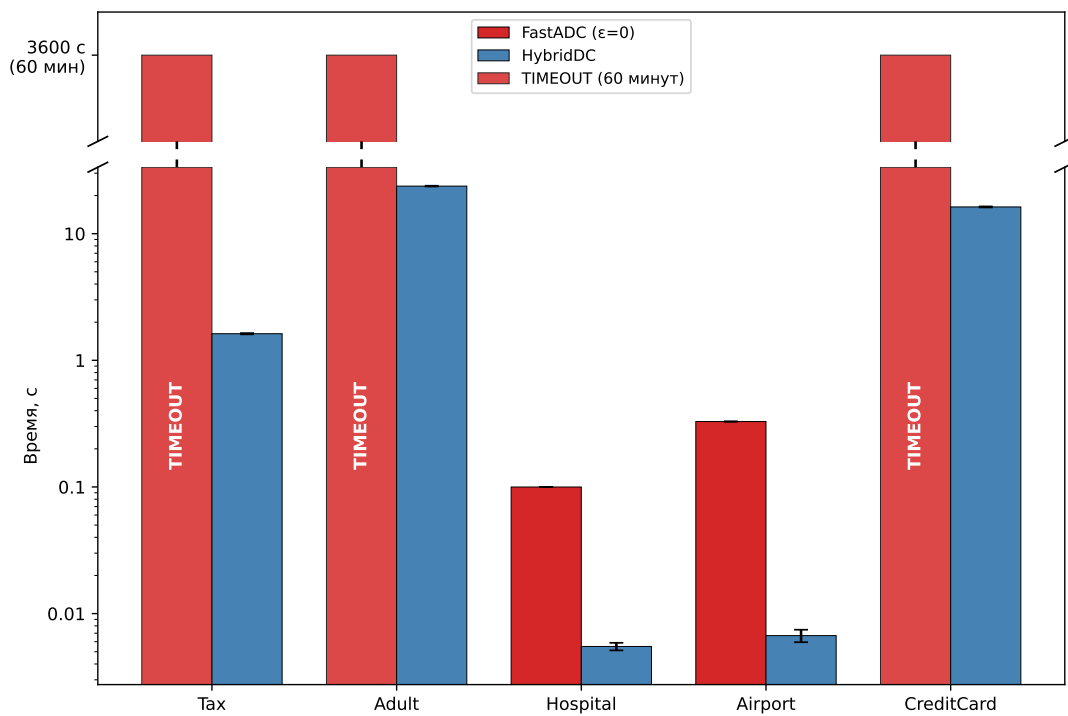


Рис. 9: Время перечисления покрытий: сравнение с FastADC при нулевом пороге ошибки.

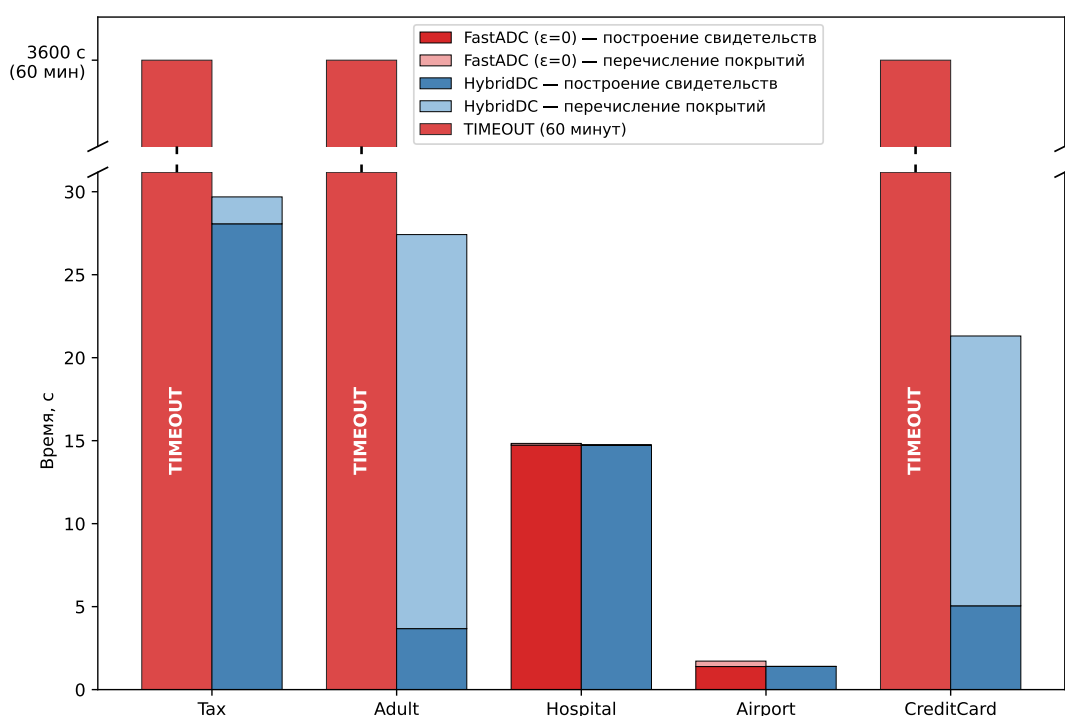


Рис. 10: Суммарное время выполнения: сравнение с FastADC при нулевом пороге ошибки. Время представлено линейной шкалой. Каждая колонка разбита на два участка: нижний — время построения набора свидетельств, верхний — время перечисления покрытий.

Использование алгоритма поиска приближённых ЗО для точного случая не оказалось устойчивым решением. Из пяти рассмотренных наборов данных FastADC завершил работу только на Hospital и Airport, где число найденных точных ЗО относительно невелико и составляет 565 и 2484 соответственно. На наборах Tax, Adult и CreditCard, содержащих 185 561, 1 450 420 и 147 773 ЗО соответственно, FastADC не завершил работу за отведённые 60 минут.

На двух наборах данных, где FastADC завершил работу, суммарное ускорение составило 1,01 раза на Hospital и 1,23 раза на Airport. Малое ускорение объясняется тем, что на этих наборах почти всё время выполнения занимает построение набора свидетельств, а данный этап у FastADC и HybridDC реализован одинаково.

При этом на этапе перечисления покрытий преимущество HybridDC выражено значительно сильнее. На Hospital время этого этапа умень-

шилось с 99,90 мс до 5,50 мс, что соответствует ускорению в 18,16 раза. На Airport время перечисления покрытий уменьшилось с 328,40 мс до 6,70 мс, что соответствует ускорению в 49,01 раза. В среднем по двум завершившимся наборам данных ускорение этапа перечисления покрытий составило 33,59 раза.

Таким образом, на малых наборах данных выигрыш HybridDC по суммарному времени ограничен тем, что основную часть времени занимает общий для обоих алгоритмов этап построения НС. Однако на наборах данных с большим числом точных ЗО FastADC при $\epsilon = 0$ не завершает работу за заданное время, тогда как HybridDC успешно обрабатывает эти наборы.

Более подробно с численными результатами можно ознакомиться в таблице 7. В таблице приведены отношения средних значений по десяти запускам: ускорение построения набора свидетельств, ускорение перечисления покрытий и ускорение суммарного времени выполнения. Минимальные значения ускорения выделены светло-красным цветом, максимальные значения — светло-зелёным. Для наборов данных, на которых FastADC не завершил работу, точное ускорение по суммарному времени неизвестно. В таблице для таких наборов приведена нижняя оценка ускорения, вычисленная из ограничения времени 60 минут и среднего времени работы HybridDC.

Таблица 7: Ускорение HybridDC относительно FastADC при $\epsilon = 0$.

Набор	# ЗО	Построение НС	Поиск покрытий	Общее время
Tax	185 561	$\times 1,00$	TIMEOUT	$> \times 121,26$
Adult	1 450 420	$\times 1,00$	TIMEOUT	$> \times 131,30$
Hospital	565	$\times 1,00$	$\times 18,16$	$\times 1,01$
Airport	2 484	$\times 1,00$	$\times 49,01$	$\times 1,23$
CreditCard	147 773	$\times 1,00$	TIMEOUT	$> \times 168,95$
Среднее	—	$\times 1,00$	$\times 33,59$	$\times 1,12$

5.3. Выводы

Таким образом, на поставленные исследовательские вопросы можно ответить следующим образом.

- **RQ1:** *Насколько быстрее предложенное решение по сравнению с оригинальной реализацией ECP/ HEI-P?*

Предложенное решение быстрее оригинальной реализации на всех рассмотренных наборах данных. Среднее ускорение по суммарному времени выполнения составило 7,46 раза. При этом этап построения набора свидетельств ускорился в среднем в 14,02 раза, а этап перечисления покрытий — в среднем в 20,15 раза.

- **RQ2:** *Позволяет ли предложенное решение сократить потребление памяти по сравнению с оригинальной реализацией ECP/HEI-P?*

Да, предложенное решение существенно снижает максимальное потребление памяти. Средняя экономия памяти составила 9,03 раза.

- **RQ3:** *Насколько быстрее предложенное решение по сравнению с реализацией FastADC при нулевом пороге ошибки?*

На наборах Hospital и Airport, где FastADC завершил работу, суммарное ускорение HybridDC составило от 1,01 до 1,23 раза. Небольшой выигрыш по суммарному времени объясняется тем, что почти всё время на этих наборах занимает построение набора свидетельств, одинаковое для обоих алгоритмов. Однако на этапе перечисления покрытий HybridDC оказался значительно быстрее: среднее ускорение составило 33,59 раза.

- **RQ4:** *Оправдан ли предложенный подход по сравнению с существующим решением через использование FastADC при нулевом пороге ошибки?*

Да, предложенный подход оправдан. FastADC при $\epsilon = 0$ завершил работу только на двух из пяти наборов данных и не справился с

наборами, содержащими большое число точных ЗО. HybridDC, напротив, успешно обрабатывает все рассмотренные наборы данных.

Заключение

В ходе выполненной работы были достигнуты следующие результаты.

- Проведён обзор существующих алгоритмов поиска точных запрещающих ограничений: FastDC, Hydra, BFastDC, DCFinder, ADC-Miner, FastADC, ECP/HMMCS-P и ECP/HEI-P. Выбран алгоритм ECP/HEI-P на основании следующих критериев: использование гетерогенных предикатов, нахождение минимальных ограничений, наличие открытой реализации, а также сравнительная эффективность. Проведено изучение выбранного алгоритма ECP/HEI-P.
- Разработана комбинированная архитектура решения, адаптированная к платформе Desbordante: построение набора свидетельств выполняется средствами FastADC, а перечисление покрытий — с помощью алгоритма HEI-P. Такое решение использует преимущества обоих алгоритмов: эффективное построение набора свидетельств из FastADC и специализированное на поиске точных 3О перечисление покрытий.
- Спроектированное решение реализовано на платформе Desbordante и выложено в свободный доступ (open-source)³.
- Проведено экспериментальное исследование. В качестве тестовых наборов данных использовались наборы данных Tax (15 атрибутов, 99 904 строки), Adult (15 атрибутов, 32 561 строка), Hospital (15 атрибутов, 100 147 строк), Airport (12 атрибутов, 24 580 строк) и CreditCard (10 атрибутов, 30 000 строк). Эксперименты показали ускорение по времени в среднем в 7,46 раз, экономию по памяти — в среднем в 9,03 раз. Проведено сравнение с FastADC при нулевом пороге ошибки: FastADC завершил работу только на наборах Hospital и Airport; на этих наборах получено ускорение этапа перечисления покрытий в среднем в 33,59 раз.

³<https://github.com/Desbordante/desbordante-core/pull/770>

Список литературы

- [1] Abedjan Ziawasch, Golab Lukasz, Naumann Felix. Profiling Relational Data: A Survey // [The VLDB Journal](#). — 2015. — Vol. 24, no. 4. — P. 557–581.
- [2] Abedjan Ziawasch, Golab Lukasz, Naumann Felix. Profiling relational data: a survey // [The VLDB Journal](#). — 2015. — Vol. 24, no. 4. — P. 557–581. — URL: <https://doi.org/10.1007/s00778-015-0389-y>.
- [3] Approximate Denial Constraints / Ester Livshits, Alireza Heidari, Ihab F. Ilyas, Benny Kimelfeld // [Proceedings of the VLDB Endowment](#). — 2020. — Vol. 13, no. 10. — P. 1682–1695. — [2005.08540](#).
- [4] Bleifuß Tobias, Kruse Sebastian, Naumann Felix. Efficient denial constraint discovery with hydra // [Proc. VLDB Endow.](#) — 2017. — Vol. 11, no. 3. — P. 311–323. — URL: <https://doi.org/10.14778/3157794.3157800>.
- [5] Chu Xu, Ilyas Ihab F., Papotti Paolo. Discovering denial constraints // [Proc. VLDB Endow.](#) — 2013. — Vol. 6, no. 13. — P. 1498–1509. — URL: <https://doi.org/10.14778/2536258.2536262>.
- [6] Chu Xu, Ilyas Ihab F., Papotti Paolo. [Holistic data cleaning: Putting violations into context](#) // 2013 IEEE 29th International Conference on Data Engineering (ICDE). — 2013. — P. 458–469.
- [7] [Cleaning inconsistencies in information extraction via prioritized repairs](#) / Ronald Fagin, Benny Kimelfeld, Frederick Reiss, Stijn Vansummenen // Proceedings of the 33rd ACM SIGMOD-SIGACT-SIGART Symposium on Principles of Database Systems. — PODS '14. — New York, NY, USA : Association for Computing Machinery, 2014. — P. 164–175. — URL: <https://doi.org/10.1145/2594538.2594540>.

- [8] [Conditional Functional Dependencies for Data Cleaning](#) / Philip Bohannon, Wenfei Fan, Floris Geerts et al. // 2007 IEEE 23rd International Conference on Data Engineering. — 2007. — P. 746–755.
- [9] [Data cleaning: Overview and emerging challenges](#) / Xu Chu, Ihab F. Ilyas, Sanjay Krishnan, Jiannan Wang // Proceedings of the 2016 International Conference on Management of Data. — SIGMOD '16. — New York, NY, USA : ACM, 2016. — P. 1727–1732. — URL: <https://dl.acm.org/doi/10.1145/2882903.2912574>.
- [10] [Desbordante: A Framework for Exploring Limits of Dependency Discovery Algorithms](#) / Maxim Strutovskiy, Nikita Bobrov, Kirill Smirnov, George Chernishev // 2021 29th Conference of Open Innovations Association (FRUCT). — 2021. — P. 344–354.
- [11] [Desbordante: From Benchmarking Suite to High-Performance Science-Intensive Data Profiler](#) / George A. Chernishev, Mikhail A. Polyntsov, Anton I. Chizhov et al. // Proceedings of the 8th International Conference on Data Science and Management of Data (CODS-COMAD 2024). — Association for Computing Machinery, 2025. — P. 234–243. — URL: <https://doi.org/10.1145/3703323.3703725>.
- [12] Fan Wenfei, Geerts Floris, Wijsen Jef. Determining the Currency of Data // [ACM Trans. Database Syst.](#) — 2012. — Vol. 37, no. 4. — 46 p. — URL: <https://doi.org/10.1145/2389241.2389244>.
- [13] Fast approximate denial constraint discovery / Renjie Xiao, Zijing Tan, Haojin Wang, Shuai Ma // [Proc. VLDB Endow.](#) — 2022. — Vol. 16, no. 2. — P. 269–281. — URL: <https://doi.org/10.14778/3565816.3565828>.
- [14] Functional dependency discovery: an experimental evaluation of seven algorithms / Thorsten Papenbrock, Jens Ehrlich, Jannik Marten et al. // [Proc. VLDB Endow.](#) — 2015. — Vol. 8, no. 10. — P. 1082–1093. — URL: <https://doi.org/10.14778/2794367.2794377>.

- [15] HoloClean: holistic data repairs with probabilistic inference / Theodoros Rekatsinas, Xu Chu, Ihab F. Ilyas, Christopher Ré // [Proc. VLDB Endow.](#) — 2017. — Vol. 10, no. 11. — P. 1190–1201. — URL: <https://doi.org/10.14778/3137628.3137631>.
- [16] Jakob Wenzel, Rhineland Jason, Moldovan Dean. pybind11 – Seamless operability between C++11 and Python. — 2017. — <https://github.com/pybind/pybind11>.
- [17] Marques Filho Sergio Luiz. [Discovering Denial Constraints Using Boolean Patterns](#) // Companion of the 2023 International Conference on Management of Data. — SIGMOD '23. — New York, NY, USA : Association for Computing Machinery, 2023. — P. 281–283. — URL: <https://doi.org/10.1145/3555041.3589392>.
- [18] Pena Eduardo H. M., Porto Fabio, Naumann Felix. Fast Algorithms for Denial Constraint Discovery // [Proc. VLDB Endow.](#) — 2022. — Vol. 16, no. 4. — P. 684–696. — URL: <https://doi.org/10.14778/3574245.3574254>.
- [19] Pena Eduardo H. M., de Almeida Eduardo Cunha. BFASTDC: A Bit-wise Algorithm for Mining Denial Constraints // Database and Expert Systems Applications / Ed. by Sven Hartmann, Hui Ma, Abdelkader Hameurlain et al. — Cham : Springer International Publishing, 2018. — P. 53–68.
- [20] Pena Eduardo H. M., de Almeida Eduardo C., Naumann Felix. Discovery of approximate (and exact) denial constraints // [Proc. VLDB Endow.](#) — 2019. — Vol. 13, no. 3. — P. 266–278. — URL: <https://doi.org/10.14778/3368289.3368293>.
- [21] Rapidash: Efficient Detection of Constraint Violations / Zifan Liu, Shaleen Deep, Anna Fariha et al. // [Proc. VLDB Endow.](#) — 2024. — Vol. 17, no. 8. — P. 2009–2021. — URL: <https://doi.org/10.14778/3659437.3659454>.

- [22] Википедия. Гиперпоточность. — <https://ru.wikipedia.org/wiki/Гиперпоточность>. — 2026. — Дата обращения: 11.03.2026.
- [23] Википедия. Одновременная многопоточность. — https://ru.wikipedia.org/wiki/Одновременная_многопоточность. — 2026. — Дата обращения: 11.03.2026.
- [24] Морозко Иван Дмитриевич. Реализация алгоритма поиска запрещающих ограничений в платформе Desbordante. — 2025. — https://github.com/spbu-se/mag_practices_2023-2025/blob/main/ПИ/Морозко%20Иван%20Дмитриевич/semester_2/Morozko_report.pdf.
- [25] Морозко Иван Дмитриевич. Реализация многопоточной версии алгоритма FastADC в платформе Desbordante. — 2026. — https://github.com/spbu-se/mag_practices_2024-2026/blob/main/ПИ/Морозко%20Иван%20Дмитриевич/semester_3/Morozko-report.pdf.